

Relatório LAB 3: Arquitetura Multiciclo e Memória

Integrantes do Grupo:

Artur Nunes de Souza - 241002485

Eduardo Oliveira Vieira - 242010024

Filipe Nobre Nunes - 242023523

Guilherme Silva Cavalcante - 242001508

Rafael Mileo Moreira Krauss Guimarães - 242024763

24 de novembro de 2025

1.1 Implementação da Arquitetura Von Neumann

Implementamos a arquitetura Von Neumann unificando instruções e dados numa única memória. Um multiplexador de endereços seleciona entre o acesso à parte da memória reservada aos dados e ao espaço reservado às instruções. Isso é basicamente determinado pelo 28º bit do endereço: se ele for 1, o endereço corresponde à memória de dados; se for 0, corresponde à memória de instruções. O multiplexador seguirá essa lógica também: ao querer gravar um dado na memória de dados ou de instruções, ele assume os valores 1 e 0 respectivamente.

Segue o código em Verilog implementado:

```
1 // Instanciação das memórias e lógica de seleção
2 ramI MemC (
3     .address(wIouD[11:2]),
4     .clock(clockMem),
5     .data(B),
6     .wren(EscreveMem & ~wIouD[28]),
7     .q(Instr)
8 );
9
10 ramD MemD (
11     .address(wIouD[11:2]),
12     .clock(clockMem),
13     .data(B),
14     .wren(EscreveMem & wIouD[28]),
15     .q(MemData)
16 );
17
18 assign rmem = wIouD[28] ? MemData : Instr;
```

Listing 1: Implementação da Memória Unificada

1.2 Adaptação para Ciclos de Memória

Como o acesso à memória necessita de 2 ciclos de clock para ser concretizado fato que, no Uniciclo, foi contornado a partir da divisão do clock para que o clock da memória fosse mais rápido do que o clock do processador, no Multiciclo tal alteração não se faz necessária nativamente, haja vista que cada etapa é realizada em um ciclo de clock separado. No entanto, a precaução que deve ser tomada é garantir que a instrução mais lenta possa ser operada em um único ciclo lógico de acesso.

Para contornar o problema do bloco de memória IP necessitar de 2 ciclos de clock para um acesso, a solução implementada foi alterar a Máquina de Estados Finitos (FSM) do Controle. Analisando o diagrama de estados, identificamos que três estados realizam acesso à memória:

- **Estado 0:** Lê da memória de instruções.
- **Estado 3:** Lê da memória de dados.
- **Estado 5:** Escreve na memória de dados.

A solução consiste em duplicar cada um desses estados. Ao fazer isso, a Máquina de Estados é forçada a gastar dois ciclos de clock completos (um no estado original e outro na sua réplica) sempre que um acesso à memória for necessário, garantindo assim que o requisito de tempo da memória IP seja satisfeito.

1.3 Estrutura da FSM Modificada

Respeitando o diagrama proposto e as alterações feitas para comportar adequadamente as instruções de acesso à memória, o seguinte código em Verilog foi desenvolvido:

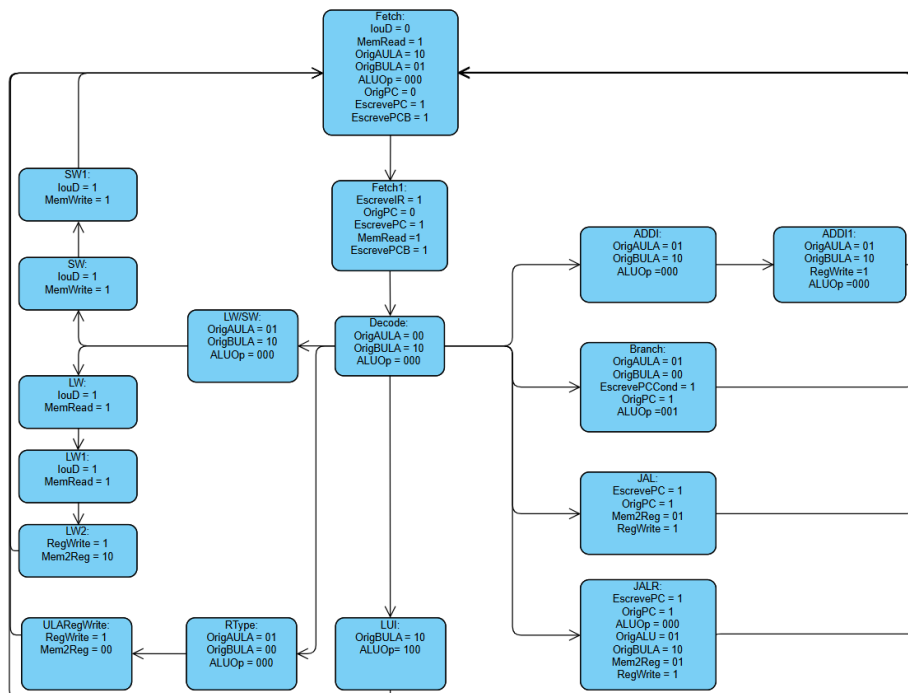


Figura 1: Diagrama ilustrando a relação entre clock do processador e memória.

Abaixo, o código completo do Bloco de Controle adaptado:

```

1 // Bloco de Controle do Multiciclo
2
3 module Control_MULTI(
4
5     input                iCLK,iRST,                //clock e reset
6     input                [6:0] Opcode,                //identifica qual tipo de
instrução
7     output                oEscreveIR,                //habilita o registrador de
instruções
8     output                oEscrevePC,                //habilita escrita no PC
9     output                oEscrevePCCond,            //identifica se o beq e
verdadeiro
10    output                oEscrevePCBack,            //salva o endereço caso
necessite de retorno (jal ou jalr), se n att
11    output [1:0]          oOrigAULA,                //1 operando da ULA (00 =
Vem do PCBack // 01 = Vem do Banco de Registradores(rs1) // 010 = Vem
do PC)
12    output [1:0]          oOrigBULA,                //2 operando da ULA (00 =
Vem do Banco de Registradores(rs2) // 001 = 4 (Pra fazer PC+ 4) //
010 = Vem do Gerador de Imediatos)
13    output [1:0]          oMem2Reg,                //De onde vem o dado a ser
escrito no banco de registradores (00 = saída da ULA (tipo I e tipo R
) // 01 = PCBack[retorno do jal jalr] // 10 = Registrador de Dado da
Memória). Ele segura a instrução ate o momento da escrita.
14    output                oOrigPC,                //De onde vem PC (0 = PC + 4
// 1 = SaídaULA (jal ou jjalr))
15    output                oIouD,                //Indica o motivo da
consulta na memória, se busca uma instrução ou um dado (0 = instrução
(procura a instrução na posição de PC) // 1 = dado(saída da ula pra
pegar o offset))
16    output                oRegWrite,                //Habilita escrita no banco
de registradores (tipo-r, load e jal jalr)
17    output                oMemWrite,                //Habilita escrita na
memoria (sw)
18    output                oMemRead,                //Habilita leitura da
memória (lw e na leitura da instrução)
19    output [1:0]          oALUOp,                //Le o Funct3 e Funct7 da
instrução, pra ver se e add,sub etc (00 = adicao // 01 = subtração //
10 = analisar funct // 11 = lui).
20    output [4:0]          oEstado                //Facilitar visualização dos
estados na forma de onda
21 );
22
23 'include "Parametros.v"
24 reg [4:0] pr_state; // present state
25 wire [4:0] nx_state; //next_state
26
27 localparam ST_FETCH = 5'd0;
28 localparam ST_FETCH1 = 5'd1;
29 localparam ST_DECODE = 5'd2;
30 localparam ST_LWSW = 5'd3;
31 localparam ST_LW = 5'd4;
32 localparam ST_LW1 = 5'd5;
33 localparam ST_LW2 = 5'd6;
34 localparam ST_SW = 5'd7;
35 localparam ST_SW1 = 5'd8;
36 localparam ST_RTYPE = 5'd9;

```

```

37     localparam ST_ULAREGWRITE = 5'd10;
38     localparam ST_BRANCH      = 5'd11;
39     localparam ST_LUI         = 5'd12;
40     localparam ST_ADDI        = 5'd13;
41     localparam ST_ADDI1       = 5'd14;
42     localparam ST_JAL         = 5'd15;
43     localparam ST_JALR        = 5'd16;
44
45 assign oEstado = pr_state; //facilitar a visualização na forma de onda
46
47 //a cada ciclo de Clock
48 always @ (posedge iCLK or posedge iRST)
49     begin
50         if (iRST)
51             pr_state <= ST_FETCH; //se resetou, volta pro estado inicial
52         else
53             pr_state <= nx_state; // próximo estado
54     end
55
56
57 always @(*)
58
59     case (pr_state)
60         //estado somente para acesso de memoria
61         ST_FETCH: //0
62             begin
63                 oEscreveIR      <= 1'b0;
64                 oEscrevePC      <= 1'b0;
65                 oEscrevePCCond  <= 1'b0;
66                 oEscrevePCBack <= 1'b0;
67                 oOrigAULA       <= 2'b00; //Se tiver dando errado,
68                 mudar pra ficar igual ao fetch1
69                 oOrigBULA       <= 2'b00; // Se tiver dando errado,
70                 mudar para ficar igual ao fetch1
71                 oMem2Reg        <= 2'b00;
72                 oOrigPC         <= 1'b0;
73                 oIouD           <= 1'b0;
74                 oRegWrite       <= 1'b0;
75                 oMemWrite       <= 1'b0;
76                 oMemRead        <= 1'b1; // Só acessa a memória e não
77                 faz mais nada
78                 oALUOp          <= 2'b00;
79                 nx_state        <= ST_FETCH1;
80             end
81
82         ST_FETCH1: //1
83             begin
84                 oEscreveIR      <= 1'b1; // escreve a instrução lida
85                 da memória
86                 oEscrevePC      <= 1'b1; // o PC sempre deve ser
87                 atualizado após um estado, seja por PC + 4 ou PC imm, n fica na mesma
88                 instrucao
89                 oEscrevePCCond  <= 1'b0; // nao e beq, nao ha
90                 necessidade de verificar
91                 oEscrevePCBack <= 1'b1; // n e desvio, salva PC + 4
92                 oOrigAULA       <= 2'b10; // Poe PC na entrada da ULA
93                 oOrigBULA       <= 2'b01; // Poe 4 na entrada da ULA
94                 oMem2Reg        <= 2'b00; // Poe a saída da ULA (PC +

```

```

4) pra salvar no Banco de Registradores
88         oOrigPC          <= 1'b0;      // Fala pro controle que e
PC + 4
89         oIouD            <= 1'b0;      // Indica que está buscando
uma instrução
90         oRegWrite        <= 1'b0;      // Nao tem rd
91         oMemWrite        <= 1'b0;      // Nao e sw
92         oMemRead         <= 1'b1;      // Garante os 2 cicclos
93         oALUOp           <= 2'b00;     // Indica soma pra ULA (PC +
4)
94         nx_state         <= ST_DECODE;
95     end
96
97     ST_DECODE: // 2
98     begin
99         oEscreveIR        <= 1'b0;      // Nao escreve nada na
memoria de instrucao, vai apenas identificala
100        oEscrevePC        <= 1'b0;      // Nao atualiza o PC
101        oEscrevePCCond <= 1'b0;      // Nao atualiza o PC
102        oEscrevePCBack <= 1'b0;      // Nao atualiza o PC
103        oOrigAULA         <= 2'b00;     // Pega a instrução atual (
talvez precise mudar???)
104        oOrigBULA         <= 2'b10;     // Gerador de imediatos,
pega os OPCODEs?? Não tenho ctz
105        oMem2Reg          <= 2'b00;     // Não tá trabalhando com
escrita
106        oOrigPC          <= 1'b0;      // Nao e jal
107        oIouD            <= 1'b0;      // Tanto Faz
108        oRegWrite        <= 1'b0;      // Na decodificação não
escreve
109        oMemWrite        <= 1'b0;      // Na decodificação não
escreve
110        oMemRead         <= 1'b0;      // Ja leu da memória na
etapa anterior
111        oALUOp           <= 2'b00;     // Nao sei pq e soma
// Durante a analise do opcode
112        case (Opcode)
113            OPC_LOAD:     nx_state      <= ST_LWSW;
114            OPC_STORE:     nx_state      <= ST_LWSW;
115            OPC_RTYPE:     nx_state      <= ST_RTYPE;
116            OPC_BRANCH:    nx_state      <= ST_BRANCH;
117            OPC_JAL:       nx_state      <= ST_JAL;
118            OPC_LUI:       nx_state      <= ST_LUI;
119            OPC_OPIMM:     nx_state      <= ST_ADDI;
120            default:       nx_state      <= ST_FETCH;
121        endcase
122    end
123
124
125    ST_LWSW: //3
126    begin
127        oEscreveIR        <= 1'b0;      // Nao escreve nada na
memoria de instrucao
128        oEscrevePC        <= 1'b0;      // Nao escreve nada no PC
129        oEscrevePCCond <= 1'b0;      // Nao e branch
130        oEscrevePCBack <= 1'b0;      // Nao muda endereço de
retorno
131        oOrigAULA         <= 2'b01;     // Pega o registrador para
pegar ou escrever o valor

```

```

132         oOrigBULA      <= 2'b10;    // Calcula o offset
133         oMem2Reg        <= 2'b00;    // Segura a instrucao que
tava na saida da ULA pra identificar o opcode
134         oOrigPC         <= 1'b0;    // Nao mexe com o PC
135         oIouD           <= 1'b0;    // Esse estado e somente a
distincao do sw/lw, entao analisa a instrucao para pegar o opcode
136         oRegWrite       <= 1'b0;    // Nao sabe se e load ainda
137         oMemWrite       <= 1'b0;    // Nao sabe se e store ainda
138         oMemRead        <= 1'b0;    // Ainda nao sabe qual e a
instrucao
139         oALUOp          <= 2'b00;    // Pega a saida da ULA que é
o rs1 + offset do immgen
140         // Analisaa o opcode
141         case (Opcode)
142             OPC_LOAD:      nx_state      <= ST_LW;
143             OPC_STORE:    nx_state      <= ST_SW;
144             default:      nx_state      <= ST_FETCH;
145         endcase
146     end
147
148     ST_LW: // 4
149     begin
150         oEscreveIR      <= 1'b0;    // Nao escreve na memoria de
instruca
151         oEscrevePC      <= 1'b0;    // Nao escreve no PC
152         oEscrevePCCond <= 1'b0;    // Nao e branch
153         oEscrevePCBack <= 1'b0;    // Nao escreve no PC
154         oOrigAULA      <= 2'b01;    // Tanto faz, já sabemos o
registrador a ser operado, repetindo o outro esestado
155         oOrigBULA      <= 2'b10;    // Tanto faz, já sabemos o
offset, repetindo o outro esestado
156         oMem2Reg        <= 2'b00;    // Ainda nao escreve nada,
primmeiro deve acessar
157         oOrigPC         <= 1'b0;    // Nao importa o PC agora
158         oIouD           <= 1'b1;    // A consulta a memmoria
esta sendo feita para buscar um dado
159         oRegWrite       <= 1'b0;    // Ainda nao escreve pois
nao pegamos o dado ainda
160         oMemWrite       <= 1'b0;    // Nao precisa escrever nada
na memoria
161         oMemRead        <= 1'b1;    // Load le da memoria
162         oALUOp          <= 2'b00;    // Tanto faz, ja tinhamos
pego o rs1 + offset, repetindo o outro esestado
163         nx_state        <= ST_LW1;  // Garante os 2 ciclos
pra mexer na memoria exigido pelo quartus
164     end
165
166     ST_LW1: // 5
167     begin
168         oEscreveIR      <= 1'b0;    // Nao escreve na memoria de
instruca
169         oEscrevePC      <= 1'b0;    // Nao escreve no PC
170         oEscrevePCCond <= 1'b0;    // Nao e branch
171         oEscrevePCBack <= 1'b0;    // Nao escreve no PC
172         oOrigAULA      <= 2'b01;    // Tanto faz, já sabemos o
registrador a ser operado, repetindo o outro esestado (tlvz mudar p 0)
173         oOrigBULA      <= 2'b10;    // Tanto faz, já sabemos o
offset, repetindo o outro esestado (tlvz mudar p 0)

```

```

174         oMem2Reg          <= 2'b00;    // Ainda nao escreve nada,
primmeiro deve acessar
175         oOrigPC           <= 1'b0;    // Nao importa o PC agora
176         oIouD             <= 1'b1;    // A consulta a memmoria
esta sendo feita para buscar um dado
177         oRegWrite         <= 1'b0;    // Ainda nao escreve pois
nao pegamos o dado ainda
178         oMemWrite         <= 1'b0;    // Nao precisa escrever nada
na memoria
179         oMemRead          <= 1'b1;    // Load le da memoria
180         oALUOp            <= 2'b00;    // Tanto faz, ja tinhamos
pego o rs1 + offset, repetindo o outro esestado
181         nx_state          <= ST_LW2;    // load word e o mais
demorado, pois acessa a memoria depois escreve no banco de regs,
entao ncessita de outro estado
182     end
183
184     // Esse estado escreve no Banco de Registradores
185     ST_LW2: // 6
186     begin
187         oEscreveIR         <= 1'b0;    // Nao escreve na memoria de
instruc
188         oEscrevePC         <= 1'b0;    // Nao escreve nada no PC
189         oEscrevePCCond     <= 1'b0;    // Nao e branch
190         oEscrevePCBack     <= 1'b0;    // Nao mexe com PC
191         oOrigAULA          <= 2'b01;    // Tanto faz, repetindo (
tlvz mudar p 0)
192         oOrigBULA          <= 2'b10;    // Tanto faz, repetindo (
tlvz mudar p 0)
193         oMem2Reg           <= 2'b10;    // Indica que o que vai ser
escrito no banco de registradores veio da memoria
194         oOrigPC            <= 1'b0;    // Nao mexemos com PC
195         oIouD              <= 1'b0;    // Tanto faz, já pegamos o
dado da memoria
196         oRegWrite          <= 1'b1;    // Habilita a escrita do
dado no Banco de Registradores
197         oMemWrite          <= 1'b0;    // Nao le nada
198         oMemRead           <= 1'b0;    // A leitura foi no passo
anterior
199         oALUOp             <= 2'b00;    // Tanto faz, ja tinhamos
pego o rs1 + offset, repetindo o outro esestado
200         nx_state           <= ST_FETCH; // Volta ao estado
inicial
201     end
202
203     ST_SW: // 7
204     begin
205         oEscreveIR         <= 1'b0;    // Nao esrceve na memoria de
instrucao, so no de dados
206         oEscrevePC         <= 1'b0;    // Nao escreve no PC
207         oEscrevePCCond     <= 1'b0;    // Nao e beq
208         oEscrevePCBack     <= 1'b0;    // Nao mexe com PC
209         oOrigAULA          <= 2'b01;    // Tanto faz, no swlw já fez
o rs1 + offset, mantendo (tlvz mudar p 0)
210         oOrigBULA          <= 2'b10;    // Tanto faz, no swlw já fez
o rs1 + offset, mantendo (tlvz mudar p 0)
211         oMem2Reg           <= 2'b00;    // Nao faz escrita no banco
de registradores, tanto faz

```

```

212         oOrigPC          <= 1'b0;      // Nao mexe com o PC
213         oIouD            <= 1'b1;      // A consulta na memoria
esta procurando um dado
214         oRegWrite        <= 1'b0;      // Nao vamos escrever no
Bando de Registradores
215         oMemWrite        <= 1'b1;      // Vamos escrever na memoria
216         oMemRead         <= 1'b0;      // Nao vamos ler nada da
memoria
217         oALUOp           <= 2'b00;     // Tanto faz, já sabemos o
endereço de rs1 + imm que veio do lsw
218         nx_state          <= ST_SW1;
219     end
220
221     ST_SW1: // 8
222     begin
223         oEscreveIR        <= 1'b0;      // Nao escreve na memoria de
instrucao, so no de dados
224         oEscrevePC        <= 1'b0;      // Nao escreve no PC
225         oEscrevePCCond    <= 1'b0;      // Nao e beq
226         oEscrevePCBack    <= 1'b0;      // Nao mexe com PC
227         oOrigAULA         <= 2'b01;     // Tanto faz, no swlw já fez
o rs1 + offset, mantendo (tlvz mudar p 0)
228         oOrigBULA         <= 2'b10;     // Tanto faz, no swlw já fez
o rs1 + offset, mantendo (tlvz mudar p 0)
229         oMem2Reg          <= 2'b00;     // Nao faz escrita no banco
de registradores, tanto faz
230         oOrigPC          <= 1'b0;      // Nao mexe com o PC
231         oIouD            <= 1'b1;      // A consulta na memoria
esta procurando um dado
232         oRegWrite        <= 1'b0;      // Nao vamos escrever no
Bando de Registradores
233         oMemWrite        <= 1'b1;      // Vamos escrever na memoria
234         oMemRead         <= 1'b0;      // Nao vamos ler nada da
memoria
235         oALUOp           <= 2'b00;     // Tanto faz, já sabemos o
endereço de rs1 + imm que veio do lsw
236         nx_state          <= ST_FETCH;
237     end
238
239     ST_RTYPE: // 9
240     begin
241         oEscreveIR        <= 1'b0;      // Nao escreve na memoria de
instrucao, so no de dados
242         oEscrevePC        <= 1'b0;      // Nao escreve no PC
243         oEscrevePCCond    <= 1'b0;      // Nao e branch
244         oEscrevePCBack    <= 1'b0;      // Nao mexe no PC
245         oOrigAULA         <= 2'b01;     // Coloca o rs1 para o
calcula
246         oOrigBULA         <= 2'b00;     // Coloca o rs2 para o
calcula
247         oMem2Reg          <= 2'b00;     // Indica que o dado a ser
escrito vem da ULA
248         oOrigPC          <= 1'b0;      // Nao mexe com PC
249         oIouD            <= 1'b0;      // Tanto faz, nao estamos
consultando com memoria
250         oRegWrite        <= 1'b0;      // Ainda nao escreve
251         oMemWrite        <= 1'b0;      // Nao acessamos a memoria
252         oMemRead         <= 1'b0;      // Nao acessamos a memoria

```



```

253         oALUOp          <= 2'b10;    // Indicamos que deve
analisar o funct3 e funct 7 pra saber se e add, sub etc
254         nx_state        <= ST_ULAREGWRITE;
255     end
256
257     ST_ULAREGWRITE:// 10
258     begin
259         oEscreveIR        <= 1'b0;    // Nao escreve na memoria de
instrucao, so dados
260         oEscrevePC        <= 1'b0;    // Nao mexe com PC
261         oEscrevePCCond    <= 1'b0;    // Nao mexe com PC
262         oEscrevePCBack    <= 1'b0;    // Nao mexe com PC
263         oOrigAULA         <= 2'b01;    // Indica que usaremos rs1 (
tanto faz, ja pegamos no ultimo)
264         oOrigBULA         <= 2'b00;    // Indica que usaremos rs2 (
tanto faz, ja pegamos no ultimo)
265         oMem2Reg          <= 2'b00;    // O dado a ser escrito vem
da ula
266         oOrigPC           <= 1'b0;    // Nao mexe com PC
267         oIouD             <= 1'b0;    // Nao estamos consultando a
memoria
268         oRegWrite         <= 1'b1;    // Vaamos escrever no banco
de registradores
269         oMemWrite         <= 1'b0;    // Nao acessamos a memoria
270         oMemRead          <= 1'b0;    // Nao acessamos a memoria
271         oALUOp           <= 2'b10;    // se der ruim mude pra 00
272         nx_state          <= ST_FETCH;
273     end
274
275     ST_BRANCH: //11
276     begin
277         oEscreveIR        <= 1'b0;    // Nao escreve na memoria de
instrucao
278         oEscrevePC        <= 1'b0;    // nao e no estagio de
branch q o pc e rreesccrito
279         oEscrevePCCond    <= 1'b1;    // analisa a condicao
280         oEscrevePCBack    <= 1'b0;    // nao e nesse estaagio que
o pc e atualizado
281         oOrigAULA         <= 2'b01;    // usa o rs1 pra verificar
igualdade
282         oOrigBULA         <= 2'b00;    // ussa o rs2 pra verificar
a igualdade
283         oMem2Reg          <= 2'b00;    // vai escrever a saida da
ula
284         oOrigPC           <= 1'b1;    // indica que o pc vai sair
da ula
285         oIouD             <= 1'b0;    // Nao ha consulta na
memoria
286         oRegWrite         <= 1'b0;    // Nao escreve no
registrador
287         oMemWrite         <= 1'b0;    // Nao escreve nada na
memoria
288         oMemRead          <= 1'b0;    // Nao faz leitura da
memoria
289         oALUOp           <= 2'b01;    // Indica que deve ser feito
uma subtacao
290         nx_state          <= ST_FETCH;
291     end

```

```

292     ST_LUI: // 12
293     begin
294         oEscreveIR      <= 1'b0;      // Nao escreve na memoria
295     de instrucao
296         oEscrevePC      <= 1'b0;      // Nao mexe no PC
297         oEscrevePCCond  <= 1'b0;      // Nao e beq
298         oEscrevePCBack  <= 1'b0;      // Nao mexe com pc
299         oOrigAULA       <= 2'b00;     // Tanto faz, so precisa do
300     imediato
301         oOrigBULA       <= 2'b10;     // Qual numero vai ser usado
302         oMem2Reg        <= 2'b00;     // Escreve a saida da ULA no
303     rd
304         oOrigPC         <= 1'b0;      // Nao mexe com PC
305         oIouD           <= 1'b0;      // Nao mexe na memoria
306         oRegWrite       <= 1'b1;      // Habilita a escrita no
307     Banco de registradores
308         oMemWrite       <= 1'b0;      // Nao acessa a memoria
309         oMemRead        <= 1'b0;      // Nao le nada da memoria
310         oALUOp          <= 2'b11;     // faz o lui
311         nx_state        <= ST_FETCH;
312     end
313
314     ST_ADDI: // 13
315     begin
316         oEscreveIR      <= 1'b0;      // Nao escreve na memoria de
317     instrucao
318         oEscrevePC      <= 1'b0;      // Nao escreve no PC
319         oEscrevePCCond  <= 1'b0;      // Não e beq
320         oEscrevePCBack  <= 1'b0;      // nao mexe compc
321         oOrigAULA       <= 2'b01;     // Rs1 pra somar o imediato
322         oOrigBULA       <= 2'b10;     // Soma o imediato do addi
323         oMem2Reg        <= 2'b00;     // Pega a saida da ULA
324         oOrigPC         <= 1'b0;      // Nao mexe no pc
325         oIouD           <= 1'b0;      // Nao mexe na memoria
326         oRegWrite       <= 1'b0;      // Ainda nao escreve como no
327     rtype
328         oMemWrite       <= 1'b0;      // Nao acessa a memoria
329         oMemRead        <= 1'b0;      // Nao acessa a memoria
330         oALUOp          <= 2'b00;     // realiza uma soma
331         nx_state        <= ST_ADDI1;
332     end
333
334     ST_ADDI1: // 14
335     begin
336         oEscreveIR      <= 1'b0;      // Nao mexe pc
337         oEscrevePC      <= 1'b0;      // Nao escreve no PC
338         oEscrevePCCond  <= 1'b0;      // Nao mexe no pc
339         oEscrevePCBack  <= 1'b0;      // Nao mexe no ppcc
340         oOrigAULA       <= 2'b01;     // Mantem por estabilidade
341         oOrigBULA       <= 2'b10;     // Mantem por estabilidade
342         oMem2Reg        <= 2'b00;     // Pega a saida da ULA
343         oOrigPC         <= 1'b0;      // Nao mexe no pc
344         oIouD           <= 1'b0;      // Nao mexe na memoria
345         oRegWrite       <= 1'b1;      // Agora ativa a escrita
346         oMemWrite       <= 1'b0;      // Nao acessa a memoria
347         oMemRead        <= 1'b0;      // Nao acessa a memoria
348         oALUOp          <= 2'b00;     // mantem

```

```

344         nx_state          <= ST_FETCH;
345     end
346
347     ST_JAL: // 15
348     begin
349         oEscreveIR          <= 1'b0;    // Nao atualiza a instrucao
350         lida
351         oEscrevePC          <= 1'b1;    // Muda o PC para a nova
352         posicao
353         oEscrevePCCond      <= 1'b0;    // Nao e beq
354         oEscrevePCBack      <= 1'b0;    // Nao muda o endereco de
355         retorno, apenas de ida
356         oOrigAULA           <= 2'b00;    // Ja foi pre calculado no
357         decode (tlvz mudar pra 0)
358         oOrigBULA           <= 2'b10;    // Ja foi pre calculado no
359         decode (tlvz mudar pra 0)
360         oMem2Reg             <= 2'b01;    // Vai salvar o retorno
361         no Banco de Registradoress
362         oOrigPC             <= 1'b1;    // O PC vai pegar a saidad
363         a ULA
364         oIouD                <= 1'b0;    // Nao mexe na memoria
365         oRegWrite            <= 1'b1;    // Vai salvar o endereço de
366         retorno
367         oMemWrite            <= 1'b0;    // Nao mexe na memoria
368         oMemRead             <= 1'b0;    // Nao acessa a memoria
369         oALUOp               <= 2'b00;    // Mantem a soma
370         nx_state             <= ST_FETCH;
371     end
372
373     ST_JALR: //16
374     begin
375         oEscreveIR          <= 1'b0;    // Nao atualiza instrucao
376         lida
377         oEscrevePC          <= 1'b1;    // Escreve no PC
378         oEscrevePCCond      <= 1'b0;    // Nao e beq
379         oEscrevePCBack      <= 1'b0;    // Nao muda o endereco de
380         rettorno
381         oOrigAULA           <= 2'b01;    // usa o rs1 pra registrar
382         argumento
383         oOrigBULA           <= 2'b10;    // Imediato pra onde vai
384         pular
385         oMem2Reg             <= 2'b01;    // Vai salvar o retorno
386         no banco de registradores
387         oOrigPC             <= 1'b1;    // PC vem da ULA
388         oIouD                <= 1'b0;    // Nao acessa a memoria
389         oRegWrite            <= 1'b1;    // Salva o retorno
390         oMemWrite            <= 1'b0;    // nao mexe na memoria
391         oMemRead             <= 1'b0;    // nao mexe na memoria
392         oALUOp               <= 2'b00;    // indica soma do endereco
393         nx_state             <= ST_FETCH;
394     end
395 endcase
396 endmodule

```

Listing 2: Bloco de Controle do Multiciclo

1.4 Análise de Timing e Simulação

1.4 a)

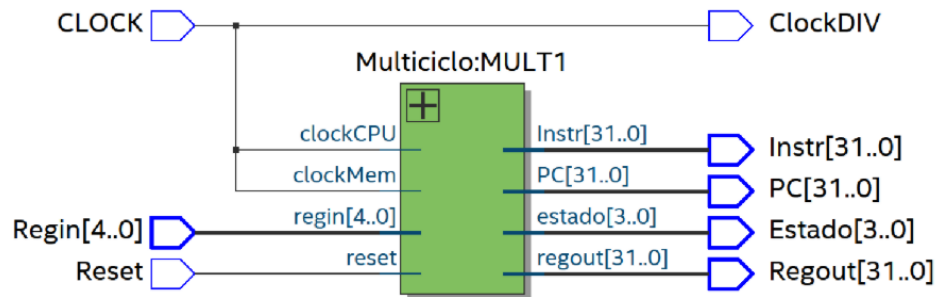


Figura 2: Visão Geral

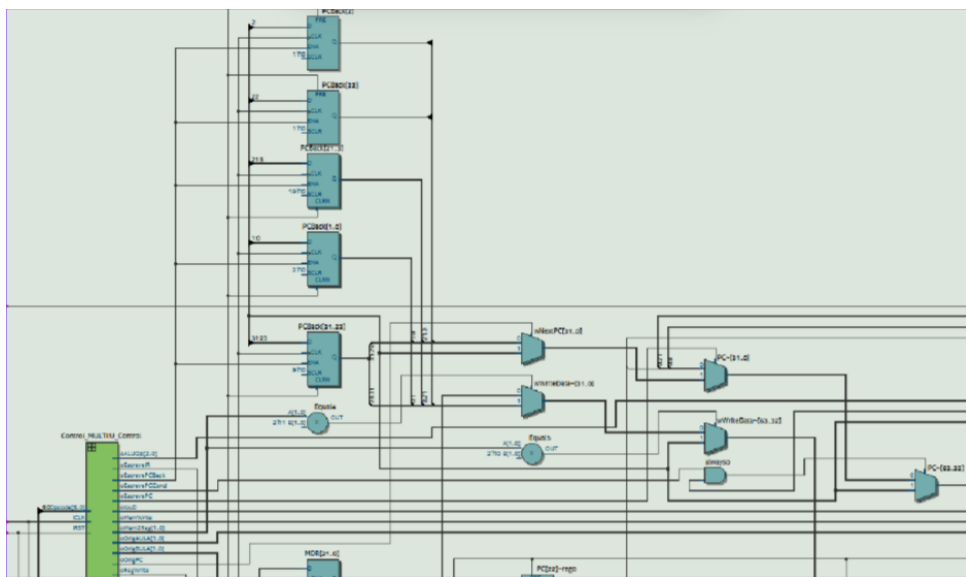


Figura 3: Controle do Multiciclo e seus muxes

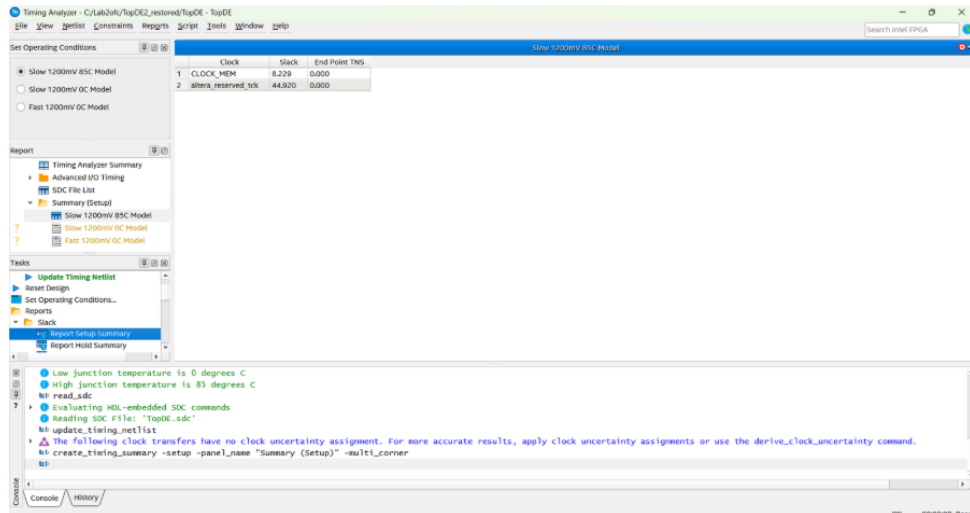


Figura 6: Análise de Setup Time: Slack positivo.

O *Slack de Hold* está positivo, ou seja, o sinal mais rápido ainda espera 0.310ns depois do clock descer.

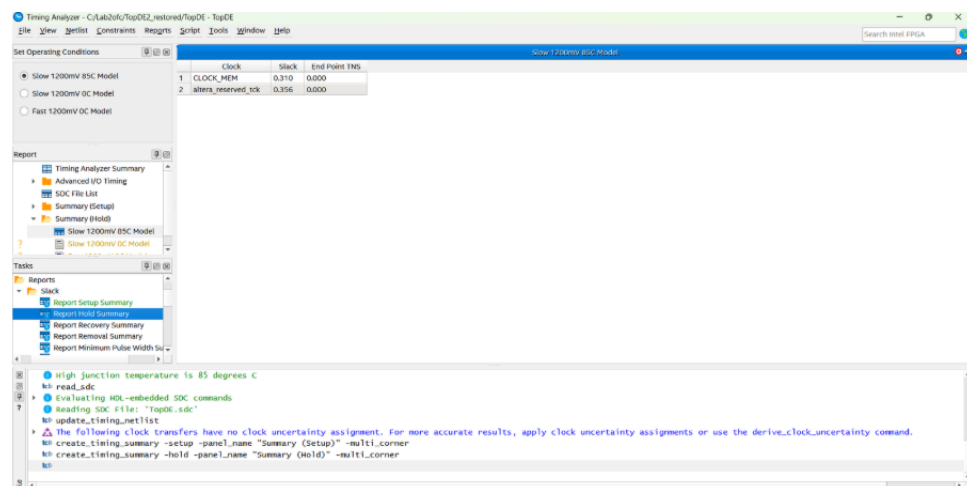


Figura 7: Análise de Hold Time: Slack positivo.

1.4 c) Sequências de Instruções

```

1 DEPTH = 16384;
2 WIDTH = 32;
3 ADDRESS_RADIX = HEX;
4 DATA_RADIX = HEX;
5 CONTENT
6 BEGIN
7 00000000 : 100101b7;    % 7:    lui gp,0x10010    # descomentar para rodar
           no Rars %
8 00000001 : 100101b7;    % 8:    li gp, 0x10010000 %
9 00000002 : 00018193;    % 8:    %
10 00000003 : 0001a303;    % 9: MAIN: lw t1,0(gp)    %
11 00000004 : 77700393;    % 10:   addi t2,zero,0x777 %
12 00000005 : 007372b3;    % 11:   and t0,t1,t2 %
13 00000006 : 007362b3;    % 12:   or t0,t1,t2 %

```

```

14 00000007 : 006382b3; % 13: add t0,t2,t1 %
15 00000008 : 406382b3; % 14: sub t0,t2,t1 %
16 00000009 : 007322b3; % 15: slt t0,t1,t2 %
17 0000000a : 0063a2b3; % 16: slt t0,t2,t1 %
18 0000000b : 00028663; % 17: beq t0,zero, PULA %
19 0000000c : eee00293; % 18: addi t0,zero,0xFFFFFEEE %
20 0000000d : 00c0006f; % 19: j FIM %
21 0000000e : 00c000ef; % 20: PULA: jal PROC %
22 0000000f : ccc00293; % 21: addi t0,zero,0xFFFFFCCC %
23 00000010 : 0000006f; % 22: FIM: j FIM %
24 00000011 : 07f00293; % 24: PROC: li t0,127 %
25 00000012 : 0051a223; % 25: sw t0,4(gp) %
26 00000013 : 0001a283; % 26: lw t0,0(gp) %
27 00000014 : 0041a283; % 27: lw t0,4(gp) %
28 00000015 : 00008067; % 28: ret %
29 END;

```

Listing 3: Instruções a serem executadas

I. Sequência: LUI → ADDI → LW

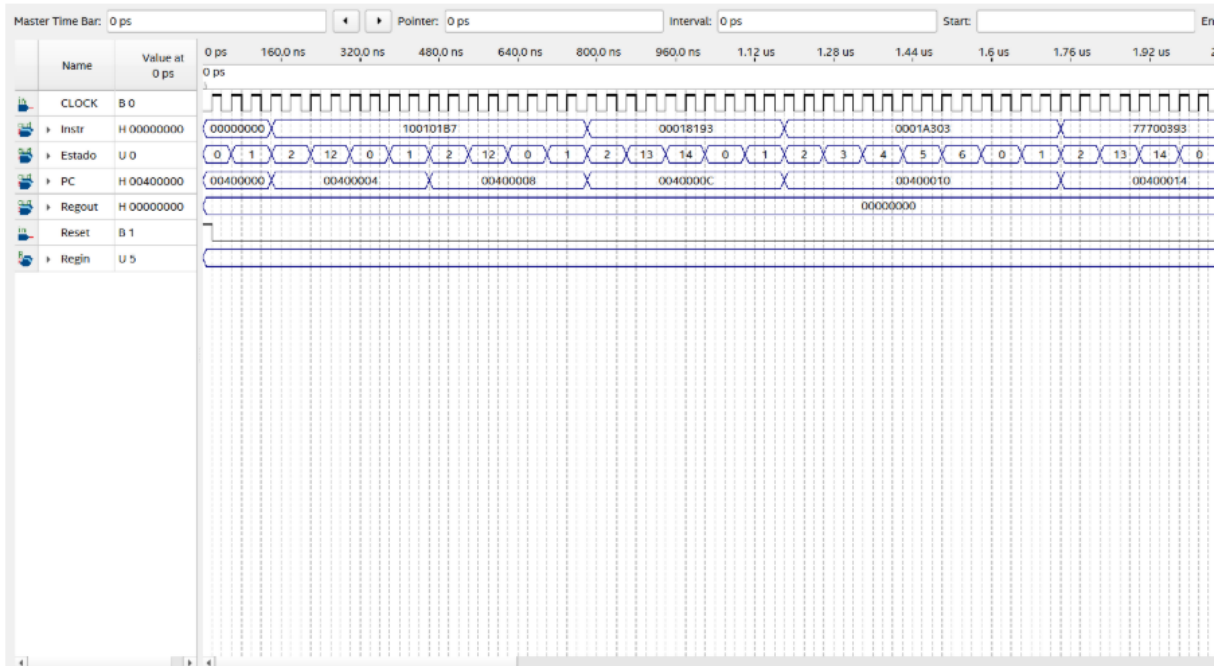


Figura 8: Simulação da sequência LUI → ADDI → LW.

II. Sequência: AND → OR → ADD → SUB → SLT → SLT

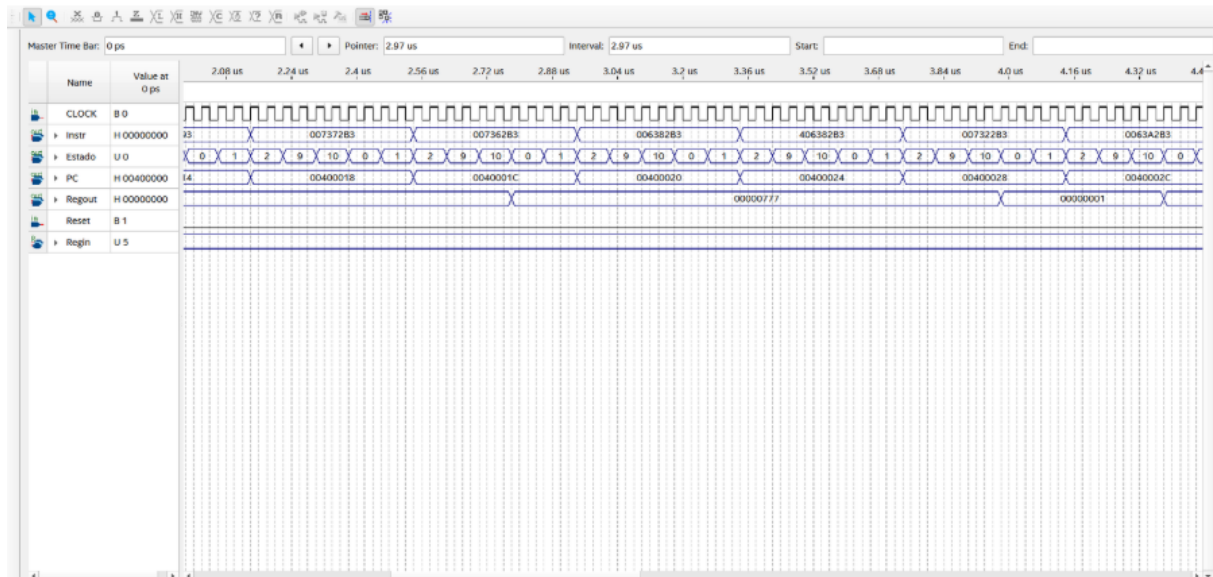


Figura 9: Simulação da sequência de operações lógicas e aritméticas.

III. Sequência: BEQ → ADDI → JAL → ADDI → SW → LW

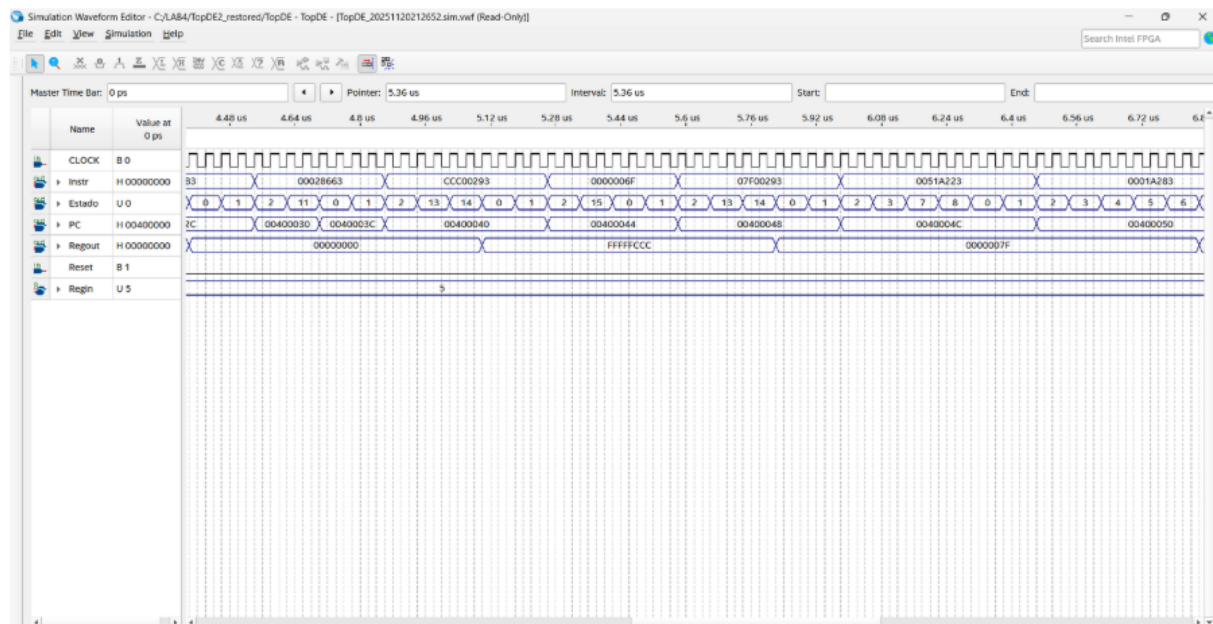


Figura 10: Simulação da sequência BEQ → ADDI → JAL → ADDI → SW → LW.

Visão Geral:

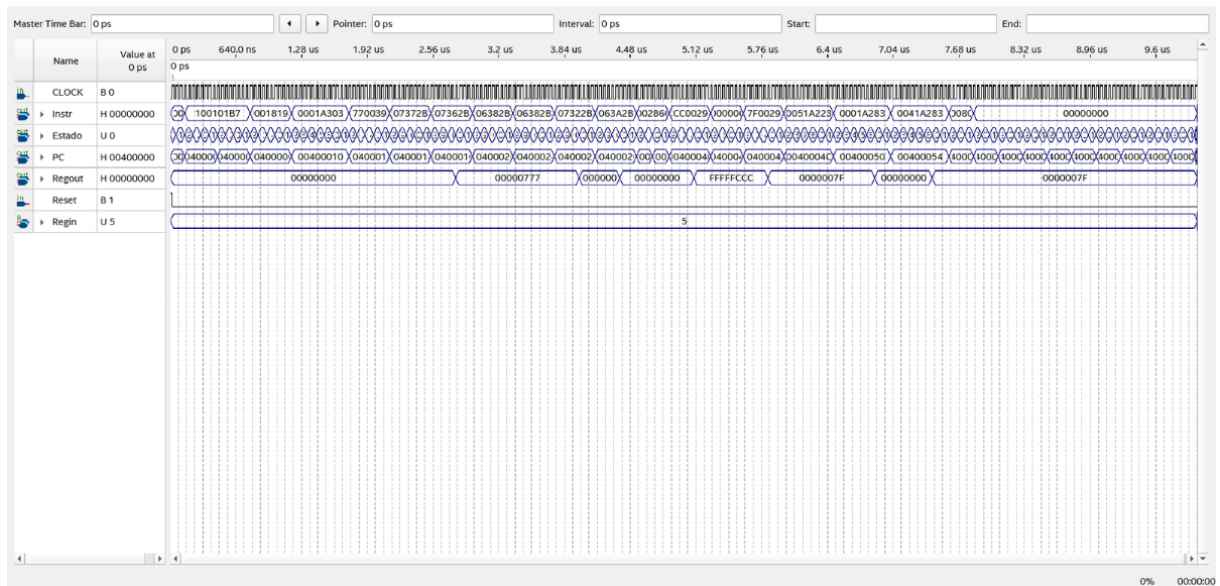


Figura 11: Visão geral das formas de onda.

1.4 d) Simulação com 250 MHz

	Fmax	Restricted Fmax	Clock Name	Note
1	96.04 MHz	96.04 MHz	altres_reserved_tck	
2	1207.73 MHz	250.0 MHz	clk	(lim_n)

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as if the rising and falling edges are scaled along with FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack reports for sign-off analysis.

Figura 12: Configuração da simulação para 250 MHz.

Abaixo seguem os resultados obtidos na simulação com frequência de 250 MHz:

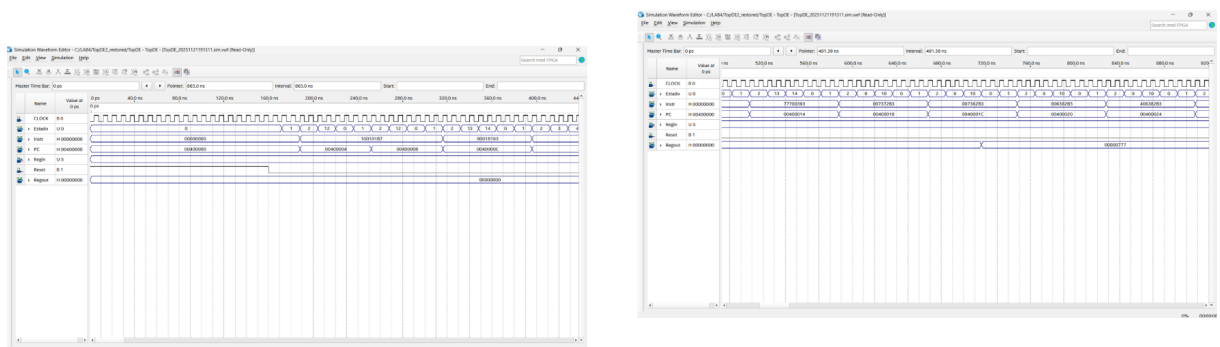


Figura 13: Detalhe simulação 2.

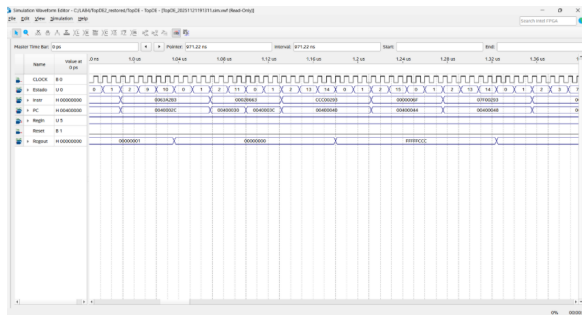


Figura 14: Detalhe simulação 3.

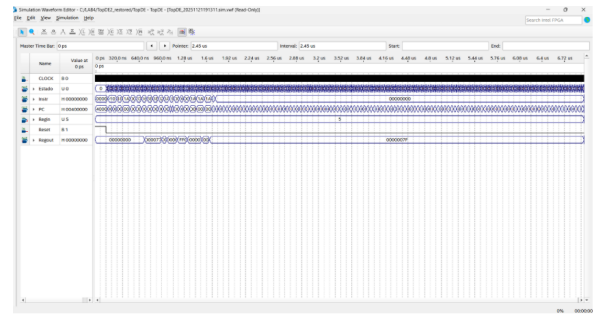


Figura 15: Detalhe simulação 4.

Acesse o video: <https://youtu.be/ECJvUiA3Zg0>